Yes. I'll make this as a **complete Compiler Design master-notes set**, not just a short revision sheet.

I checked your uploaded semester study guide: your **CS24301 Compiler Design syllabus contains 47 topics across M1–M5**. The breakdown is **7 + 10 + 9 + 9 + 12 = 47 topics**.

One important point: I searched your uploaded files for **CS24301/CD PYQs**, but I could not find an actual Compiler Design question paper among the uploaded documents. So I **won't falsely label invented questions as PYQs**. I'll mark questions as **PYQ-style / exam-important** unless you upload the CD papers.

🧠 COMPILER DESIGN — COMPLETE MASTER NOTES

How these notes are designed

For every topic, I'll use:

1. Understand → 2. Visualize → 3. Connect → 4. Recall → 5. Apply → 6. Exam answer

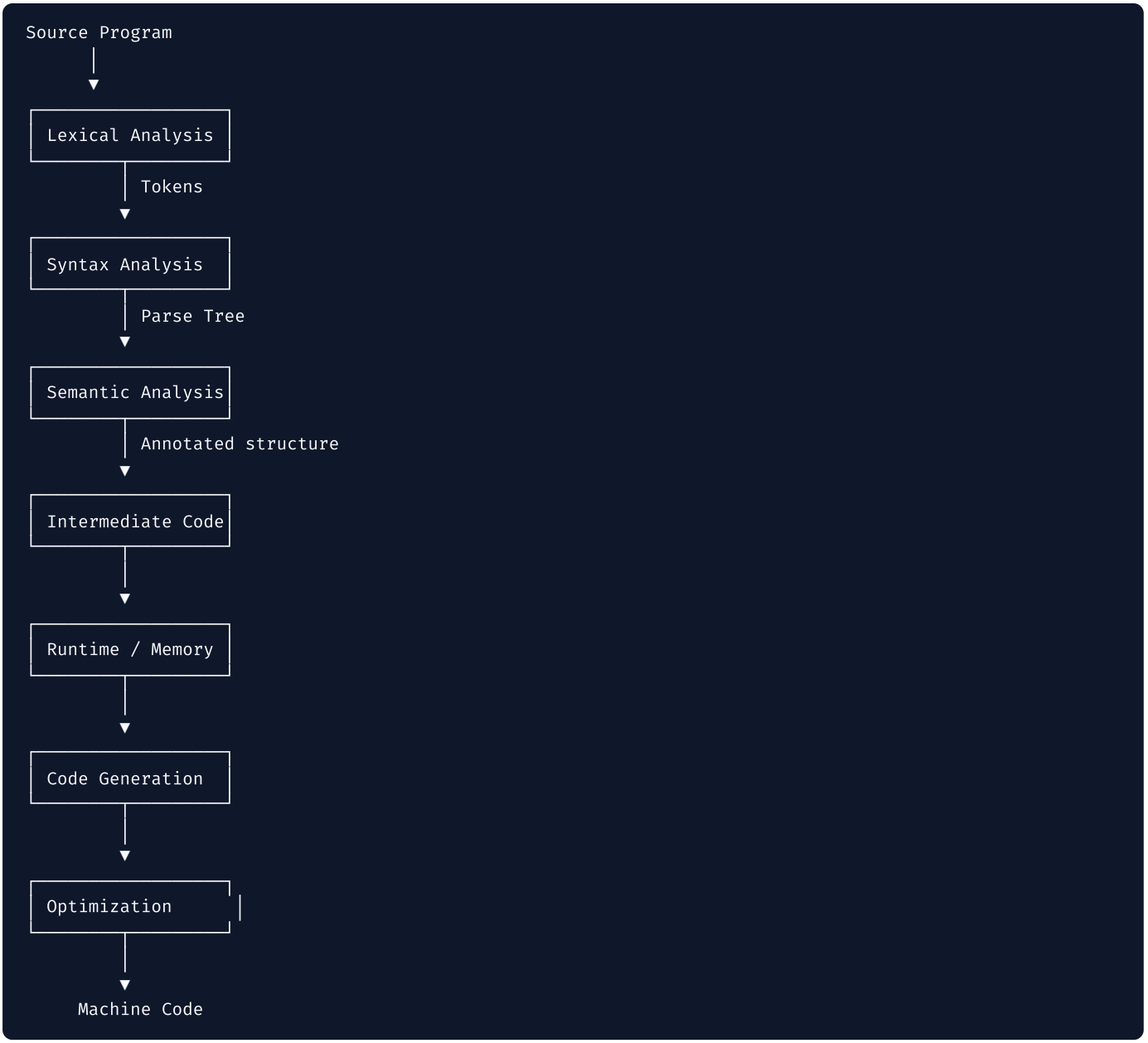
This uses the learning techniques that are actually useful for technical learning:

- **Active recall** — questions before/after concepts
- **Spaced repetition** — compact recall blocks
- **Chunking** — group related concepts
- **Dual coding** — diagrams + words
- **Elaboration** — explain why/how
- **Generation effect** — attempt before seeing solution
- **Interleaving** — mix conceptual and numerical/parser problems
- **Retrieval cues** — keywords and memory hooks
- **Feynman technique** — explain concepts simply
- **Worked examples → faded examples → independent problems**

🗺️ THE BIG PICTURE

Remember Compiler Design as:

> **Key Insight**
>
> **Characters → Tokens → Structure → Meaning → Intermediate Code → Runtime → Optimized Code**



MODULE I — LEXICAL ANALYSIS

Your syllabus has **7 topics** here.

1. Introduction to Compilers and its Cousins

Compiler

A **compiler** translates a source program written in a high-level language into an equivalent target program, usually machine/assembly code.

```
High-Level Language
        ↓
     Compiler
        ↓
Assembly / Machine Code
```

Example:

```
a = b + c;
```

might eventually become machine instructions such as:

```
LOAD R1,b
LOAD R2,c
ADD  R1,R2
STORE a,R1
```

Compiler vs Interpreter

Compiler	Interpreter
Translates program before execution	Executes source program directly/stepwise
Usually produces target code	Usually doesn't produce standalone target code
Compilation may take time	Execution starts quickly
Executable can run repeatedly	Source often needs interpretation again
Errors can be reported during compilation	Errors often appear as execution reaches them

Assembler

Assembly:

```
MOV R1, R2
```

↓

Assembler

↓

Machine code

Linker

Combines:

- object files
- libraries
- external references

into an executable.

Loader

Loads executable into memory and prepares it for execution.

Preprocessor

Performs source-level transformations such as:

```
#include
#define
```

before compilation.

Memory hook

> **Key Insight**
>
> **P-I-C-A-L**

Preprocessor

Compiler

Assembler

Linker

Loader

---

2. Structure of a Compiler

A compiler is generally divided into **front end** and **back end**.

Main phases

1. Lexical Analysis

Characters → tokens.

2. Syntax Analysis

Tokens → syntactic structure.

3. Semantic Analysis

Checks meaning and consistency.

4. Intermediate Code Generation

Produces machine-independent representation.

5. Code Optimization

Improves code.

6. Code Generation

Produces target machine code.

Two activities used by many phases

Symbol Table

Stores information about identifiers.

Example:

```
Name     Type       Scope
──────────────────────────
x        int        local
y        float      global
```

Error Handler

Detects/reports errors throughout compilation.

---

3. Lexical Analyzer

The lexical analyzer is the **first major compiler phase**.

Input:

```
int sum = a + 10;
```

Output:

```
<int>
<id,sum>
<assign>
<id,a>
<plus>
<number,10>
<semicolon>
```

Important terminology

Token

A category/class.

Example:

```
IDENTIFIER
NUMBER
PLUS
KEYWORD
```

Lexeme

Actual character sequence.

Example:

```
sum
10
int
```

Pattern

Rule describing valid lexemes.

Example:

```
identifier → letter(letter|digit)*
```

Relationship

> **Key Insight**
> **Token = category**

> **Key Insight**
> **Lexeme = actual instance**

> **Key Insight**
> **Pattern = rule**

Example:

```
count = 25;
```

Lexeme	Token
count	ID
=	ASSIGN
25	NUM
;	SEMICOLON

4. Input Buffering

Reading one character at a time directly from disk is expensive.

Therefore lexical analyzers use **buffering**.

Two-buffer scheme

Usually two pointers are maintained:

lexemeBegin

Beginning of current lexeme.

forward

Scans characters ahead.

When one buffer is exhausted, the other can be refilled.

Sentinel

A special end marker can be placed at the end of each buffer to reduce repeated boundary checking.

Why buffering?

Without buffering:

```
character → I/O → character → I/O → ...
```

With buffering:

```
large block → memory → many characters processed
```

Exam point: buffering improves lexical-analysis efficiency by reducing costly input operations.

5. Specification of Tokens

Tokens are normally specified using **regular expressions**.

Common definitions

Identifier

```
letter(letter|digit)*
```

Integer

```
digit+
```

Whitespace

```
(space|tab|newline)+
```

Relational operators

```
< | ⩽ | > | ⩾ | = | ≠
```

Regular expression operators

Operator	Meaning	
`	`	OR
concatenation	sequence	
`*`	zero or more	
`+`	one or more	
`?`	optional	
`( )`	grouping	

Example:

```
(a|b)*
```

accepts:

```
ε
a
b
ab
ba
aab
abb
...
```

6. Recognition of Tokens

Specification tells us **what is valid**.

Recognition determines whether the input actually belongs to that token class.

Typical process:

```
Regular Expression
       ↓
     NFA/DFA
       ↓
Token recognizer
       ↓
Input lexeme
```

For example:

```
identifier = [a-zA-Z][a-zA-Z0-9]*
```

Input:

```
student123
```

Accepted.

Input:

```
123student
```

Rejected as an identifier.

---

7. DFA Directly from Regular Expressions

This is a **high-value problem-solving topic**.

Your syllabus explicitly includes construction of DFA directly from regular expressions.

The standard direct method uses **syntax-tree positions**.

Core concepts

For a regular expression, augment it:

```
R#
```

where `#` is a special end marker.

Then construct a syntax tree.

Each leaf gets a position number.

Example:

```
(a|b)*abb
```

Positions:

```
a1
b2
a3
b4
b5
```

Important functions:

nullable(n)

Whether node can generate ε.

firstpos(n)

Positions that can appear first.

lastpos(n)

Positions that can appear last.

followpos(i)

Positions that can immediately follow position `i`.

Rules

Leaf

For position `i`:

```
nullable = false
firstpos = {i}
lastpos = {i}
```

ε

```
nullable = true
firstpos = ∅
lastpos = ∅
```

OR

For:

```
A | B
```

```
nullable = nullable(A) OR nullable(B)

firstpos = firstpos(A) ∪ firstpos(B)

lastpos = lastpos(A) ∪ lastpos(B)
```

Concatenation

For:

```
A B
```

```
nullable = nullable(A) AND nullable(B)
```

If A is nullable:

```
firstpos = firstpos(A) ∪ firstpos(B)
```

otherwise:

```
firstpos = firstpos(A)
```

Similarly, if B is nullable:

```
lastpos = lastpos(A) ∪ lastpos(B)
```

otherwise:

```
lastpos = lastpos(B)
```

For every `i ∈ lastpos(A)`:

```
followpos(i) += firstpos(B)
```

Kleene star

For:

```
A*
```

```
nullable = true
firstpos = firstpos(A)
lastpos = lastpos(A)
```

For every:

```
i ∈ lastpos(A)
```

add:

```
firstpos(A)
```

to `followpos(i)`.

DFA construction

Start state:

```
firstpos(root)
```

For a DFA state S and input symbol a:

```
U = union of followpos(p)
    for every p ∈ S
    where position p contains a
```

Then U becomes a DFA state.

A state is accepting if it contains the position corresponding to `#`.

Memory hook

> **Key Insight**
>
> **FLF → Follow**

Firstpos

Lastpos

Followpos

🧠 M1 ACTIVE RECALL

Don't look above while answering:

1. What is the difference between token, lexeme and pattern?
2. Why is buffering needed?
3. What is a sentinel?
4. What are the phases of a compiler?
5. What is the difference between compiler and interpreter?
6. What is the purpose of a symbol table?
7. What are nullable, firstpos, lastpos and followpos?
8. How do you construct a DFA directly from an RE?

🔥 Exam-important M1 questions

- Explain the phases/structure of a compiler.
- Differentiate compiler, interpreter, assembler, linker and loader.
- Explain lexical analyzer with token, lexeme and pattern.
- Explain input buffering and two-buffer scheme.
- Specify tokens using regular expressions.
- Explain token recognition.
- Construct DFA directly from a given regular expression.

MODULE II — SYNTAX ANALYSIS

Your syllabus has **10 topics**.

---

1. Introduction to Syntax Analysis

Lexical analysis gives:

```
tokens
```

Syntax analysis determines whether those tokens form a valid grammatical structure.

Example:

```
a + b * c
```

A parser determines its structure:

```
      +
     / \
    a   *
       / \
      b   c
```

The parser works using a **Context-Free Grammar (CFG)**.

---

2. Grammar Rewriting Transformations

Two particularly important transformations:

Left recursion

Grammar:

```
E → E + T | T
```

is left recursive.

Transform:

```
E  → T E'
E' → + T E' | ε
```

General rule

```
A → Aα | β
```

becomes:

```
A  → βA'
A' → αA' | ε
```

Left factoring

Grammar:

```
A → αβ₁ | αβ₂
```

has common prefix α.

Transform:

```
A  → αA'
A' → β₁ | β₂
```

Why?

Top-down parsers need to choose productions based on upcoming input.

Left factoring delays the decision until enough input is available.

---

3. Recursive Top-Down Parsers

A top-down parser starts from:

```
Start symbol
```

and attempts to derive the input.

```
Start
  ↓
Nonterminal
  ↓
Production
  ↓
Terminals
```

Recursive descent

Each nonterminal can correspond to a procedure.

Example:

```
E → T E'
```

may become conceptually:

```
E()
{
    T();
    Eprime();
}
```

Problem

Naive recursive descent cannot directly handle left recursion.

Example:

```
E → E + T
```

causes infinite recursion.

---

4. Non-Recursive Top-Down Parsers

A non-recursive predictive parser uses:

- input buffer
- stack
- parsing table

The parser uses:

```
M[A,a]
```

where:

- A = stack nonterminal
- a = current input token

---

5. Design of LL(1) Parser

LL(1):

- first L = scan input **Left to right**
- second L = produce **Leftmost derivation**
- 1 = use **one lookahead symbol**

FIRST

`FIRST(X)` = terminals that can begin strings derived from X.

Examples:

```
A → aB | bC
```

then:

```
FIRST(A) = {a,b}
```

If:

```
A → ε
```

then:

```
ε ∈ FIRST(A)
```

FOLLOW

`FOLLOW(A)` = terminals that can appear immediately after A.

For start symbol:

```
$ ∈ FOLLOW(S)
```

Important rules

For:

```
A → αBβ
```

add:

```
FIRST(β) - {ε}
```

to:

```
FOLLOW(B)
```

If:

```
β ⇒* ε
```

then:

```
FOLLOW(A)
```

is also added to:

```
FOLLOW(B)
```

LL(1) parsing table

For:

```
A → α
```

put production in:

```
M[A,a]
```

for every:

```
a ∈ FIRST(α)
```

If:

```
ε ∈ FIRST(α)
```

then put A → α in:

```
M[A,b]
```

for every:

```
b ∈ FOLLOW(A)
```

LL(1) grammar condition

Each table cell should contain **at most one production**.

If two productions appear in one cell:

> **Key Insight**
>
> Grammar is not LL(1).

---

6. Bottom-Up Parsers

Top-down:

```
Start → input
```

Bottom-up:

```
input → Start
```

Bottom-up parsing repeatedly performs:

```
SHIFT
REDUCE
```

This is called **shift-reduce parsing**.

---

7. Variants of LR Parsers

Main LR families:

```
LR(0)
SLR(1)
CLR(1)
LALR(1)
```

LR

Reads input left-to-right and constructs a **rightmost derivation in reverse**.

LR(0)

Uses LR(0) items.

Example:

```
A → α . β
```

The dot represents parser progress.

SLR(1)

Uses:

- LR(0) items
- FOLLOW sets

Reductions are placed using FOLLOW.

CLR(1)

Uses full LR(1) items:

```
[A → α . β, a]
```

where `a` is lookahead.

More powerful but generally larger tables.

LALR(1)

Combines compatible LR(1) states.

Advantages:

- much smaller than CLR
- more powerful than SLR for many grammars
- widely used in parser generators

Memory hook

> **Key Insight**
> S → C → L

SLR

CLR

LALR

---

8. Handling Parsing Conflicts

Shift-reduce conflict

Parser cannot decide between:

```
SHIFT
```

and

```
REDUCE
```

Classic example:

```
dangling else
```

or ambiguous arithmetic grammars.

Reduce-reduce conflict

Parser has two possible reductions.

```
R1
or
R2
```

This usually indicates a serious grammar ambiguity/problem.

---

9. Detection of Syntax Errors

Examples:

```
a + * b
```

```
if (x
```

Parser detects invalid token sequences.

Methods include:

- panic-mode recovery
- phrase-level recovery
- error productions
- global correction

---

10. Reporting of Syntax Errors

A good parser should report:

```
what went wrong
where it happened
possibly what was expected
```

Bad:

```
Syntax error
```

Better:

```
Syntax error at line 5:
expected ')' before ';'
```

🧠 M2 ACTIVE RECALL

1. Why can't LL parsers directly handle left recursion?
2. Convert:

```
A → Aα | β
```

1. What is left factoring?
2. Define FIRST.
3. Define FOLLOW.
4. Why is `$` placed in FOLLOW(start)?
5. What does LL(1) mean?
6. Difference between top-down and bottom-up parsing.
7. What is shift-reduce parsing?
8. Compare LR(0), SLR(1), CLR(1), LALR(1).
9. What is shift-reduce conflict?
10. What is reduce-reduce conflict?
11. Name syntax-error recovery techniques.

🔥 Exam-important M2 questions

- Eliminate left recursion.
- Perform left factoring.

- Calculate FIRST and FOLLOW.
- Construct an LL(1) parsing table.
- Parse a string using LL(1).
- Explain recursive descent.
- Explain predictive parsing.
- Explain shift-reduce parsing.
- Compare LR parser variants.
- Explain parsing conflicts.
- Explain syntax-error detection and recovery.

MODULE III — SEMANTIC ANALYSIS + INTERMEDIATE CODE

Your syllabus has **9 topics**.

---

1. Introduction to Semantic Analysis

Syntax answers:

> **Key Insight**
>
> **Is the structure valid?**

Semantics answers:

> **Key Insight**
>
> **Does the program make sense?**

Example:

```
int x;
x = "hello";
```

Syntax may be valid.

Semantic analysis detects:

```
type mismatch
```

Typical checks:

- type compatibility
- declaration before use
- scope
- function arguments
- return types
- array indexing
- identifier consistency

2. Syntax-Directed Definitions — SDD

An SDD associates **attributes** with grammar symbols and semantic rules with productions.

Example:

```
E → E1 + T
```

Semantic rule:

```
E.val = E1.val + T.val
```

The grammar defines structure.

The semantic rules define meaning.

3. Syntax-Directed Translation Schemes — SDTS

In an SDTS, semantic actions are embedded directly inside productions.

Example:

```
E → E + T { print("+") }
```

The action is executed at a particular point during parsing.

SDD vs SDTS

SDD	SDTS
Semantic rules associated with productions	Actions embedded in productions
Attribute-oriented	Action-oriented
Describes computation	Specifies when action occurs

4. SDTS for Declaration Processing

Declarations provide type/symbol information.

Example:

```
int a, b, c;
```

Symbol table:

```
Name     Type
a        int
b        int
c        int
```

Semantic processing can:

1. identify type
2. identify identifiers
3. enter identifiers into symbol table
4. detect duplicate declarations
5. associate type information

---

5. Three Address Code — TAC

TAC uses instructions with at most three addresses.

Example:

```
a = b + c * d
```

becomes:

```
t1 = c * d
t2 = b + t1
a = t2
```

Common TAC forms

Binary operation

```
x = y op z
```

Unary

```
x = op y
```

Copy

```
x = y
```

Conditional jump

```
if x relop y goto L
```

Unconditional jump

```
goto L
```

Procedure call

```
param x
call f,n
```

Return

```
return x
```

---

6. Types of Attributes

Two fundamental types:

Synthesized attribute

Computed from children.

```
parent ← children
```

Example:

```
E.val = E1.val + T.val
```

Inherited attribute

Obtained from:

- parent
- siblings

```
parent/sibling → node
```

Memory hook

> **Key Insight**
> **S = bottom-up**

> **Key Insight**
> **I = information coming in**

---

7. Type Checking for Expressions

Compiler verifies whether operations are type-compatible.

Example:

```
int + int → int
```

But:

```
int + string
```

is generally invalid unless the language defines a conversion.

Type conversion

Widening

```
int → float
```

usually safe.

Narrowing

```
float → int
```

may lose information.

Type equivalence

Two common ideas:

Name equivalence

Types considered equal only when declared from the same named type.

Structural equivalence

Types considered equivalent when structures match.

---

8. Intermediate Code Generation for Assignment Statements

Example:

```
a = b + c * d
```

TAC:

```
t1 = c * d
t2 = b + t1
a = t2
```

For array assignment, address calculation is also required.

---

9. Translation of Multi-Dimensional Array References

This is a very important numerical/conversion topic.

Suppose:

```
A[i][j]
```

For row-major storage:

```
address(A[i][j])
=
base(A)
+ ((i × number_of_columns) + j) × width
```

For a 2–D array:

```
A[rows][columns]
```

with zero-based indexing.

Example:

```
A[10][20]
```

Address:

```
base + (i×20 + j)×w
```

General n-dimensional idea

Convert multi-dimensional index into a **linear offset**.

For row-major:

```
offset =
(((i1 × D2 + i2) × D3 + i3) ... )
```

then:

```
address = base + offset × element_width
```

🧠 M3 ACTIVE RECALL

1. Syntax vs semantics?
2. What is SDD?
3. What is SDTS?
4. SDD vs SDTS?
5. What is TAC?
6. Write TAC for:

```
x = a + b*c
```

1. Synthesized vs inherited attributes?
2. What is type checking?
3. Widening vs narrowing conversion?
4. How is `A[i][j]` translated?

🔥 **Exam-important M3**

- Explain semantic analysis.
- Explain SDD with example.
- Explain SDTS with example.
- Explain declaration processing.
- Generate TAC for arithmetic expressions.
- Explain synthesized and inherited attributes.
- Explain type checking.
- Generate intermediate code for assignments.
- Translate multidimensional array references.

MODULE IV — INTERMEDIATE CODE + RUNTIME ENVIRONMENT

Your syllabus contains **9 topics**.

---

1. Complete Evaluation of Boolean Expressions

For:

```
A && B
```

both operands need to be evaluated according to the language's boolean semantics.

Conceptually:

```
evaluate A
if false → result false
evaluate B
result = B
```

---

2. Partial Evaluation of Boolean Expressions

This is **short-circuit evaluation**.

For:

```
A && B
```

if A is false:

```
B is not evaluated
```

For:

```
A || B
```

if A is true:

```
B is not evaluated
```

Memory hook

```
AND → false stops
OR  → true stops
```

---

3. Translation of Control Flow Constructs

Important constructs:

```
if
if-else
while
do-while
for
```

Example:

```
if (a < b)
    x = 1;
else
    x = 2;
```

TAC:

```
if a < b goto L1
goto L2

L1:
x = 1
goto L3

L2:
x = 2

L3:
```

---

4. Resolution of Forward Jumps

A forward jump targets a location that hasn't been generated yet.

Example:

```
goto L1
...
L1:
```

At the time `goto` is emitted, the address of L1 may be unknown.

So compiler can temporarily keep:

```
goto ?
```

and later fill the destination.

This idea is central to **backpatching**.

---

5. Resolution of Backward Jumps

Backward jumps point to already generated code.

Example:

```
L1:
...
goto L1
```

The target is already known.

Backward jumps are common in loops.

```
while
   ↓
condition
   ↓
body
   ↓
jump backward
```

---

6. Translation of Function Calls

Function call typically involves:

```
arguments
↓
parameter passing
↓
control transfer
↓
new activation record
↓
function execution
```

Example:

```
x = f(a,b);
```

Conceptual TAC:

```
param a
param b
call f,2
t1 = return_value
x = t1
```

7. Translation of Function Returns

Example:

```
return x;
```

Intermediate representation:

```
return x
```

Runtime must:

1. obtain return value
2. transfer control back
3. restore caller state
4. continue caller execution

8. Memory Layout of Code and Data

Typical process address space:

Common regions:

Code/Text

Executable instructions.

Static/Data

Global/static variables.

Heap

Dynamic allocation.

Stack

Function calls and automatic local data.

---

9. Activation Records

Every function call generally gets an **activation record**.

Typical structure:

Control link

Points to caller's activation record.

Access link

Helps access non-local variables in nested procedures.

Return address

Where execution resumes after function returns.

Memory hook

> **Key Insight**
>
> **P–R–C–A–S–L–T**

Parameters

Return value

Control information

Access information

Saved registers

Locals

Temporaries

🧠 M4 ACTIVE RECALL

1. What is short-circuit evaluation?
2. What happens with `A && B` when A=false?
3. What happens with `A || B` when A=true?
4. What is a forward jump?
5. What is a backward jump?
6. Why is backpatching needed?
7. What happens during a function call?
8. What are code, data, heap and stack?
9. What is an activation record?
10. Control link vs access link?

🔥 Exam-important M4

- Explain complete vs partial Boolean evaluation.
- Generate TAC for if-else.
- Generate TAC for loops.
- Explain forward and backward jumps.
- Explain backpatching.
- Translate function calls.
- Translate function returns.
- Draw memory layout.
- Draw and explain activation record.

MODULE V — CODE GENERATION & OPTIMIZATION

Your uploaded syllabus lists **12 topics** here.

These are:

1. Addresses of Code and Data in Assembly Code
2. Correlation of Assembly Code with Source Code
3. Construction of Basic Blocks
4. Control Flow Graph
5. Machine-Independent Local Optimizations
6. Machine-Independent Global Optimizations
7. Unreachable Code Elimination
8. Constant Folding
9. Constant Propagation
10. Loop-Invariant Code Motion
11. Common Subexpression Elimination
12. Dead Code Elimination

1. Addresses of Code and Data in Assembly Code

Compiler eventually maps program entities to memory/register addresses.

Example:

```
x → memory address 1000
y → memory address 1004
```

Assembly may contain symbolic references:

```
LOAD R1, x
STORE y, R1
```

The assembler/linker/loader ultimately resolve actual addresses depending on the target system and relocation model.

2. Correlation of Assembly Code with Source Code

Compiler maintains correspondence between:

```
source statement
        ↓
IR
        ↓
assembly
```

Example:

```
x = a + b;
```

↓

```
t1 = a + b
x = t1
```

↓

```
LOAD R1,a
ADD R1,b
STORE x,R1
```

This mapping is useful for:

- debugging
- optimization analysis
- source-level error reporting
- understanding generated code

3. Construction of Basic Blocks

A **basic block** is a maximal sequence of consecutive statements with:

- one entry
- one exit
- no internal branching

Finding leaders

A statement is a leader if:

Rule 1

First statement is a leader.

Rule 2

Target of a jump is a leader.

Rule 3

Statement immediately following a jump is a leader.

Example:

```
1  a = b + c
2  if a < d goto 5
3  x = y + z
4  goto 6
5  x = 0
6  print x
```

Leaders:

```
1
3
5
6
```

Basic blocks:

```
B1: 1,2
B2: 3,4
B3: 5
B4: 6
```

---

4. Control Flow Graph — CFG

A CFG represents possible flow between basic blocks.

Example:

Nodes

Basic blocks.

Edges

Possible control transfers.

Used for:

- optimization
- loop detection
- reachability
- data-flow analysis

---

5. Machine-Independent Local Optimizations

Local optimization works within a basic block.

Example:

```
t1 = a + b
t2 = t1 * 2
```

Compiler can remove unnecessary temporary operations when possible.

Common local optimizations:

- constant folding
- algebraic simplification
- local common-subexpression elimination
- copy propagation
- dead-code elimination

---

6. Machine-Independent Global Optimizations

Global optimization considers multiple basic blocks.

Examples:

- global common subexpression elimination
- global constant propagation
- loop optimization
- code motion

Local vs Global

Local	Global
One basic block	Multiple blocks
Simpler	More complex
Limited information	Uses CFG/data-flow information

---

7. Unreachable Code Elimination

Code that can never execute is removed.

Example:

```
return 5;
x = 10;
```

`x = 10` is unreachable.

Optimized:

```
return 5;
```

Another example:

```
goto L1

L2:
x = 10
```

If nothing can reach L2, eliminate it.

---

8. Constant Folding

Evaluate constant expressions at compile time.

Before:

```
x = 10 * 20
```

After:

```
x = 200
```

Another:

```
y = 4 + 5 * 2
```

↓

```
y = 14
```

Memory hook

> **Key Insight**
>
> **Folding = calculate now**

---

9. Constant Propagation

Replace a variable known to contain a constant.

Before:

```
x = 10
y = x + 5
```

After:

```
x = 10
y = 10 + 5
```

Then constant folding may produce:

```
y = 15
```

Difference

Constant folding:

```
2 + 3 → 5
```

Constant propagation:

```
x=5
y=x+2

→

y=5+2
```

---

10. Loop-Invariant Code Motion

If an expression inside a loop doesn't depend on changing loop variables, move it outside.

Before:

```
while ( ... )
{
    x = a * b;
    y++;
}
```

If `a` and `b` don't change:

```
x = a * b;

while ( ... )
{
    y++;
}
```

Benefit

Avoid repeated computation.

---

11. Common Subexpression Elimination

If the same expression is evaluated multiple times and operands haven't changed:

Before:

```
x = a + b
y = a + b
```

After:

```
t = a + b
x = t
y = t
```

Important condition

The operands must not have changed between evaluations.

---

12. Dead Code Elimination

Dead code computes something that is never used.

Example:

```
x = 10
x = 20
print x
```

The first assignment is dead:

```
x = 20
print x
```

Unreachable vs Dead

Unreachable:

> **Key Insight**
> Code cannot execute.

Dead:

> **Key Insight**
> Code may execute, but its result is never useful.

🧠 M5 ACTIVE RECALL

1. What is a basic block?
2. State the three leader rules.
3. What is a CFG?
4. Local vs global optimization?
5. Constant folding vs propagation?
6. What is unreachable code?
7. What is dead code?
8. What is common-subexpression elimination?
9. What is loop-invariant code motion?
10. Why are basic blocks important for optimization?

🧠 THE MOST IMPORTANT DIFFERENCES

These are excellent for **active-recall cards**.

Token vs Lexeme vs Pattern

```
TOKEN   → category
LEXEME  → actual text
PATTERN → rule
```

Compiler vs Interpreter

```
Compiler     → translate first
Interpreter  → execute/interpret progressively
```

Syntax vs Semantics

```
Syntax   → structure
Semantics → meaning
```

Top-down vs Bottom-up

```
Top-down   → Start → input
Bottom-up  → input → Start
```

SDD vs SDTS

```
SDD  → attributes + semantic rules
SDTS → embedded semantic actions
```

Synthesized vs Inherited

```
Synthesized → children → parent
Inherited   → parent/sibling → child
```

Complete vs Partial Boolean Evaluation

```
Complete → evaluate operands according to full expression
Partial  → short-circuit when result already known
```

Forward vs Backward Jump

```
Forward  → target not generated yet
Backward → target already generated
```

Local vs Global Optimization

```
Local  → one basic block
Global → multiple blocks
```

Constant Folding vs Propagation

```
Folding:
3 + 4 → 7

Propagation:
x=7
y=x+2

→
y=7+2
```

Unreachable vs Dead Code

```
Unreachable → cannot execute
Dead        → executes but result is useless
```

🔥 THE 47-TOPIC CHECKLIST

This is the part I want you to use as your **master progress tracker**.

M1 — 7/7

- [ ] 1. Introduction to Compilers and its Cousins
- [ ] 2. Structure of a Compiler
- [ ] 3. Lexical Analyzer
- [ ] 4. Input Buffering
- [ ] 5. Specification of Tokens
- [ ] 6. Recognition of Tokens
- [ ] 7. DFA directly from Regular Expressions

M2 — 10/10

- [ ] 8. Introduction to Syntax Analysis
- [ ] 9. Grammar Rewriting Transformations
- [ ] 10. Recursive Top-Down Parsers
- [ ] 11. Non-Recursive Top-Down Parsers
- [ ] 12. LL(1) Parser
- [ ] 13. Bottom-Up Parsers
- [ ] 14. LR Parser Variants
- [ ] 15. Parsing Conflicts
- [ ] 16. Syntax Error Detection
- [ ] 17. Syntax Error Reporting

M3 — 9/9

- [ ] 18. Semantic Analysis
- [ ] 19. SDD
- [ ] 20. SDTS
- [ ] 21. SDTS for Declaration Processing
- [ ] 22. Three Address Code
- [ ] 23. Attributes
- [ ] 24. Type Checking
- [ ] 25. Assignment Statement Translation
- [ ] 26. Multidimensional Array Translation

M4 — 9/9

- [ ] 27. Complete Boolean Evaluation
- [ ] 28. Partial Boolean Evaluation
- [ ] 29. Control Flow Translation
- [ ] 30. Forward Jumps
- [ ] 31. Backward Jumps
- [ ] 32. Function Calls
- [ ] 33. Function Returns
- [ ] 34. Memory Layout
- [ ] 35. Activation Records

M5 — 12/12

- [ ] 36. Addresses of Code/Data
- [ ] 37. Assembly ↔ Source Correlation
- [ ] 38. Basic Blocks
- [ ] 39. Control Flow Graph
- [ ] 40. Local Optimizations
- [ ] 41. Global Optimizations
- [ ] 42. Unreachable Code Elimination
- [ ] 43. Constant Folding
- [ ] 44. Constant Propagation
- [ ] 45. Loop-Invariant Code Motion
- [ ] 46. Common Subexpression Elimination
- [ ] 47. Dead Code Elimination

Total = 47/47 topics.

🎯 HOW YOU SHOULD STUDY THIS

Don't simply read the notes repeatedly. That produces familiarity, not reliable recall.

Use this cycle for **every topic**:

```
1. READ
  ↓
2. CLOSE NOTES
  ↓
3. RECALL
  ↓
4. DRAW / DERIVE
  ↓
5. SOLVE QUESTION
  ↓
6. CHECK
  ↓
7. EXPLAIN FROM MEMORY
```

Spaced repetition

For a topic learned today:

```
Day 0  → learn
Day 1  → recall
Day 3  → recall
Day 7  → recall
Day 14 → recall
Day 30 → recall
```

For **CD specifically**, don't just memorize definitions. Your highest-value retrieval practice should be:

RE → DFA

Grammar → FIRST/FOLLOW → LL(1)

Grammar → LR concepts

Expression → TAC

Array → address/TAC

Boolean/control flow → TAC

Basic blocks → CFG

Unoptimized code → optimized code

Those transformations force your brain to **generate** the answer rather than recognize it.

⚠️ One correction about the PYQs

I have **not inserted fake "PYQs."** I searched the available uploaded files and the actual CD papers were not found. The material available contains your CS24301 syllabus/checklist, but the searchable question papers I found are for other subjects. So the questions above are **exam-important/PYQ-style questions**, not claimed historical PYQs.

If you upload your **Compiler Design PYQ PDFs**, I can merge their **actual questions topic-by-topic into these notes**, identify repeated questions, and mark 🔥 **repeated** / ⭐ **high-frequency / numerical / theory / must-practice** without inventing anything.